

Assignment1

Somber Xie

September 2026

1 Problem 1: Merge Intervals

1.1 1. Specification

Input:

A vector of intervals. Each interval contains two integers `[start, end]`, where `start <= end`.

Output:

A vector containing all intervals after overlapping intervals have been merged. The returned intervals are non-overlapping and sorted by starting value.

For example:

Input:

`[[1,3], [2,6], [8,10], [15,18]]`

Output:

`[[1,6], [8,10], [15,18]]`

1.2 2. Structure

The solution is implemented in the function:

```
mergeIntervals(vector<Interval> intervals)
```

The program first checks whether every interval is valid. It then sorts the intervals by starting value. A vector named `merged` stores the final result.

The first interval is inserted into `merged`. For every remaining interval:

- If its start is less than or equal to the end of the last merged interval, the two intervals overlap and are merged.
- Otherwise, the interval is added as a new interval.

1.3 3. Pseudocode

```
MERGE-INTERVALS(intervals)
```

```
    if intervals is empty  
        return empty list
```

```
    sort intervals by start value
```

```

===== EECE 7205 Assignment #01 =====
1. Merge Intervals
2. Sort 0, 1, 2
3. Largest Number
4. Group Anagrams
5. Can Attend All Meetings
6. K Closest Points
7. Top K Frequent Values
8. Merge K Sorted Arrays
9. Kth Largest Element
10. Count Inversions
0. Exit
Choose a problem: 1
Number of intervals: 4
Enter each interval as: start end
1 3
2 6
8 10
15 18
Merged intervals: [[1, 6], [8, 10], [15, 18]]

```

Figure 1: Enter Caption

```

merged = empty list
add intervals[0] to merged

for each interval from intervals[1] to intervals[n-1]
    if interval.start <= merged.last.end
        merged.last.end =
            max(merged.last.end, interval.end)
    else
        add interval to merged
return merged

```

1.4 4. Correctness

After sorting, intervals are processed from the smallest starting value to the largest.

The **merged** vector always contains non-overlapping intervals. When a new interval overlaps with the last interval in **merged**, extending the end to the larger endpoint correctly represents both intervals.

If there is no overlap, the current interval starts after the previous merged interval ends, so it can safely be added as a new interval.

Therefore, after all intervals are processed, **merged** contains exactly the required non-overlapping intervals.

Example:

[1,3] and [2,6] overlap because $2 \leq 3$.

They are therefore merged into:

[1,6]

1.5 5. Analysis of Complexity

Sorting the intervals requires:

Time: $O(n \log n)$

The merging scan requires:

Time: $O(n)$

Therefore, the total time complexity is:

Best Case: $O(n \log n)$

Average Case: $O(n \log n)$

Worst Case: $O(n \log n)$

The result vector may contain up to n intervals.

Space Complexity: $O(n)$ including the output vector.

2 Problem 2: Sort 0, 1, and 2

2.1 1. Specification

Input:

A vector containing only the values 0, 1, and 2.

Output:

The same values sorted in the following order:

0, 0, ..., 1, 1, ..., 2, 2, ...

Two solutions are implemented:

1. Counting-based solution
2. Dutch National Flag algorithm

For example:

Input:

[2,0,2,1,1,0]

Output:

[0,0,1,1,2,2]

2.2 2. Structure

Two separate functions are used.

2.2.1 Counting Method

`sort012Counting(vector<int>& values)`

An integer array:

`count[3]`

stores how many 0s, 1s, and 2s occur.

After counting, the original vector is overwritten with the correct number of each value.

2.2.2 Dutch National Flag Method

`sort012DNF(vector<int>& values)`

Three indices are used:

- `low`: position for the next 0
- `mid`: current element being examined
- `high`: position for the next 2

The array is divided into regions containing 0s, 1s, unprocessed values, and 2s.

2.3 3. Pseudocode

2.3.1 Counting Method

`COUNTING-SORT-012(values)`

`count[0] = 0`

`count[1] = 0`

`count[2] = 0`

for each value in values

`count[value]++`

`index = 0`

for value = 0 to 2

repeat `count[value]` times

`values[index] = value`

`index++`

2.3.2 Dutch National Flag

`DUTCH-NATIONAL-FLAG(values)`

`low = 0`

`mid = 0`

`high = n - 1`

while `mid <= high`

if `values[mid] == 0`

swap `values[low]` and `values[mid]`

`low++`

`mid++`

else if `values[mid] == 1`

`mid++`

else

swap `values[mid]` and `values[high]`

`high--`

.png

Figure 2: Enter Caption

2.4 4. Correctness

For the counting solution, the algorithm first determines exactly how many values of each category exist. Writing that many 0s, followed by 1s and then 2s produces a correctly sorted array without changing the number of elements.

For the Dutch National Flag solution, during execution:

- Elements before `low` are 0.
- Elements between `low` and `mid - 1` are 1.
- Elements after `high` are 2.
- Elements between `mid` and `high` are not yet processed.

Each iteration moves one element into its correct region. When `mid > high`, there are no unprocessed elements remaining, so the entire array is sorted.

2.5 5. Analysis of Complexity

2.5.1 Counting Method

The array is scanned once for counting and once for rewriting.

Best Case: $O(n)$

Average Case: $O(n)$

Worst Case: $O(n)$

Space Complexity: $O(1)$

2.5.2 Dutch National Flag

Each element is processed at most a constant number of times.

Best Case: $O(n)$

Average Case: $O(n)$

Worst Case: $O(n)$

Space Complexity: $O(1)$

3 Problem 3: Largest Number

3.1 1. Specification

Input:

A vector of nonnegative integers.

Output:

A string representing the largest number that can be formed by concatenating all input integers.

Example:

Input:

[3,30,34,5,9]

Output:

"9534330"

3.2 2. Structure

The main function is:

```
largestNumber(const vector<int>& numbers)
```

Each integer is first converted into a string.

The strings are then sorted with a custom comparators.

For two strings *a* and *b*, the algorithm compares:

a + *b*

with:

b + *a*

If *a* + *b* is larger, then *a* should appear before *b*.

After sorting, all strings are concatenated into the final answer.

If the first sorted string is "0", the function returns "0" to avoid results such as "000".

3.3 3. Pseudocode

```
LARGEST-NUMBER(numbers)
```

```
    if numbers is empty
```

```
        return empty string
```

```
    convert every number to a string
```

```
    sort strings using comparator:
```

```
        if a + b > b + a
```

```
            place a before b
```

```
    if first string is "0"
```

```
        return "0"
```

```
    result = empty string
```

```
    for each string
```

```
        append string to result
```

```
    return result
```

3.4 4. Correctness

For every pair of values *a* and *b*, only two relative orders are possible:

ab

or

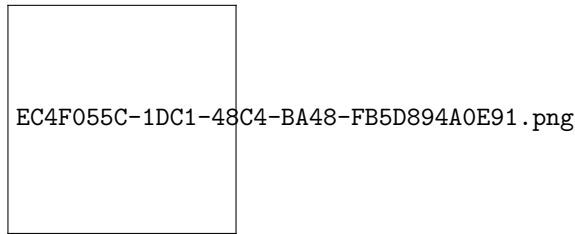


Figure 3: Enter Caption

ba

If $ab > ba$, placing a before b creates a larger combined number.

The custom comparator therefore places each pair in the order that produces the larger concatenation.

After the complete sort, concatenating all strings gives the largest possible number.

For example, comparing "9" and "34":

"934" > "349"

Therefore, 9 must appear before 34.

3.5 5. Analysis of Complexity

Let n be the number of integers and m be the maximum number of digits in an integer.

Sorting requires approximately:

$O(n \log n)$ comparisons.

Each comparison may process up to $O(m)$ characters.

Therefore:

Time Complexity: $O(n \log n * m)$

For the sorting operation:

Best Case: $O(n \log n * m)$

Average Case: $O(n \log n * m)$

Worst Case: $O(n \log n * m)$

The strings and final result require additional storage.

Space Complexity: $O(nm)$

4 Problem 4: Group Anagrams

4.1 1. Specification

Input:

A vector of strings.

Output:

A collection of groups where each group contains words that are anagrams of each other.

Words inside each group retain their original input order.

Example:

Input:

```
["eat", "tea", "tan", "ate", "nat", "bat"]
```

Possible output:

```
[ ["eat", "tea", "ate"], ["tan", "nat"], ["bat"] ]
```

4.2 2. Structure

The main function is:

```
groupAnagrams(const vector<string>& words)
```

Two main data structures are used:

```
unordered_map<string, size_t> groupIndex vector<vector<string>> groups
```

For every word, the characters are sorted to create a normalized key.

For example:

```
eat -> aet
```

```
tea -> aet
```

```
ate -> aet
```

All words having the same key belong to the same group.

`groupIndex` maps each key to its corresponding group in `groups`.

4.3 3. Pseudocode

```
GROUP-ANAGRAMS(words)
    create empty hash map groupIndex
    create empty list groups
    for each word in words
        key = word
        sort characters in key

        if key is not in groupIndex
            groupIndex[key] = groups.size
            create new group containing word
        else
            index = groupIndex[key]
            add word to groups[index]

    return groups
```

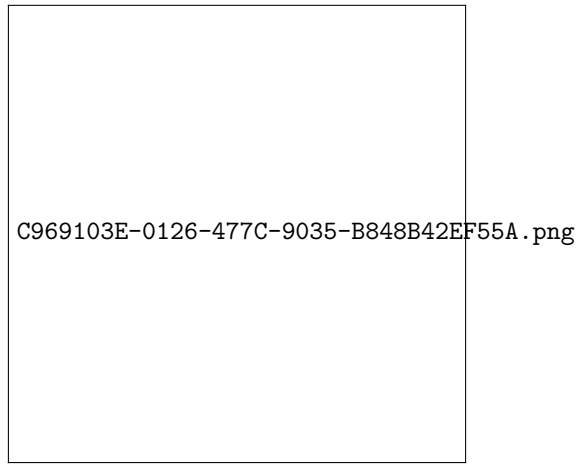



Figure 4: Enter Caption

4.4 4. Correctness

Two words are anagrams exactly when they contain the same characters with the same frequencies.

Sorting the characters of two anagrams produces the same string.

For example:

```
eat -> aet tea -> aet ate -> aet
```

Therefore, all anagrams receive the same hash-map key and are placed in the same group.

Because the words are processed in their original input order and appended to each group's vector, the original order inside every group is preserved.

4.5 5. Analysis of Complexity

Let:

- n = number of words
- m = maximum word length

Sorting each word requires:

$O(m \log m)$

For n words:

Average Time Complexity: $O(n * m \log m)$

Hash-map insertion and lookup are $O(1)$ on average.

In the theoretical worst case, hash collisions can make hash-map operations slower, although this is uncommon.

The algorithm stores all input words and their keys.

Space Complexity: $O(nm)$

5 Problem 5: Can Attend All Meetings

5.1 1. Specification

Input:

A vector of meeting intervals `[start, end]`.

Output:

A Boolean value:

- `true` if one person can attend all meetings.
- `false` if any meetings overlap.

A meeting beginning exactly when another meeting ends is allowed.

For example:

Input:

`[[0,30], [5,10], [15,20]]`

Output:

`false`

Input:

`[[7,10], [10,12]]`

Output:

`true`

5.2 2. Structure

The solution is implemented in:

```
canAttendAllMeetings(vector<Interval> meetings)
```

The meetings are sorted by start time.

The algorithm then compares each meeting with the previous meeting.

If:

```
current.start < previous.end
```

the meetings overlap.

If:

```
current.start == previous.end
```

there is no conflict.

5.3 3. Pseudocode

```
CAN-ATTEND-ALL-MEETINGS(meetings)
```

```
    sort meetings by starting time
```

```
    for i = 1 to meetings.size - 1
```

```
        if meetings[i].start
```

```
            < meetings[i - 1].end
```

```
                return false
```

```
    return true
```

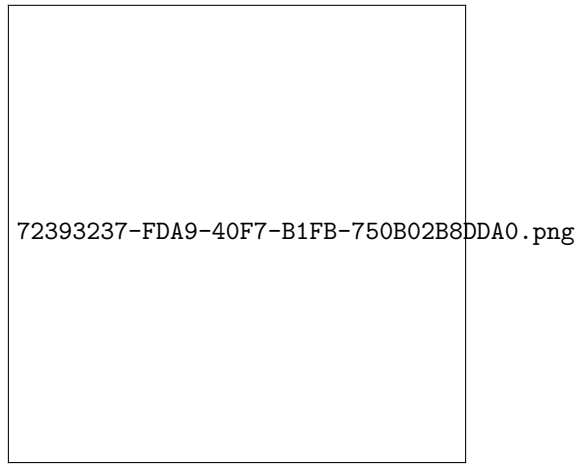


Figure 5: Enter Caption

5.4 4. Correctness

After sorting by starting time, any meeting that conflicts with an earlier meeting will create an overlap between adjacent meetings in the sorted order.

If the current meeting begins before the previous meeting ends, the two meetings overlap and attending both is impossible.

If every adjacent pair satisfies:

`current.start >= previous.end`

then every meeting begins after or exactly when the previous meeting ends.

Therefore, the person can attend every meeting.

The strict $<$ comparison correctly allows intervals such as:

$[7, 10]$ and $[10, 12]$.

5.5 5. Analysis of Complexity

Sorting the meetings requires:

$O(n \log n)$

Checking adjacent meetings requires:

$O(n)$

Therefore:

Best Case: $O(n \log n)$

Average Case: $O(n \log n)$

Worst Case: $O(n \log n)$

The sorting algorithm uses limited auxiliary storage.

Space Complexity: approximately $O(\log n)$ auxiliary space, excluding the input copy.

6 Problem 6: K Closest Points to the Origin

6.1 1. Specification

Input:

- A vector of points (x, y)
- An integer k

Output:

The k points closest to the origin (0,0).

The squared Euclidean distance is used:

$$\text{distance}^2 = x^2 + y^2$$

Using squared distance avoids an unnecessary square-root operation.

Two solutions are implemented:

1. Sorting solution
2. Heap solution

6.2 2. Structure

The helper function:

```
squaredDistance(const Point& p)
    computes the squared distance.
```

6.2.1 Sorting Solution

```
kClosestSort(vector<Point> points, int k)
```

All points are sorted by distance. The first k points are returned.

6.2.2 Heap Solution

```
kClosestHeap(const vector<Point>& points, int k)
```

A max-heap of at most k points is maintained.

The point at the top of the heap is the farthest among the currently selected points.

If a new point is closer, the farthest point is removed and replaced.

6.3 3. Pseudocode

6.3.1 Sorting Solution

```
K-CLOSEST-SORT(points, k)
    sort points by  $x^2 + y^2$ 
    return first k points
```

6.3.2 Heap Solution

```
K-CLOSEST-HEAP(points, k)
    create empty maxHeap
    for each point
        distance =  $x^2 + y^2$ 
        if heap size < k
            insert point
        else if point is closer than heap top
            remove heap top
            insert point
    move heap elements into result
    sort selected points by distance
    return result
```

6.4 4. Correctness

6.4.1 Sorting Solution

After sorting all points by increasing squared distance, the first k points must be the k closest points to the origin.

6.4.2 Heap Solution

The max-heap always stores at most k candidate points.

If the heap contains fewer than k points, the new point is inserted.

Once the heap contains k points, its top element is the farthest selected point.

If a new point is closer than the heap top, replacing the top improves the selected set.

After all points have been processed, no point outside the heap is closer than a point inside it.

Therefore, the heap contains exactly the k closest points.

6.5 5. Analysis of Complexity

6.5.1 Sorting Solution

Sorting n points requires:

Time Complexity: $O(n \log n)$

Returning k points requires $O(k)$ space.

Space Complexity: $O(k)$ for the returned result, excluding sorting overhead.

6.5.2 Heap Solution

The heap contains at most k points.

Each heap operation costs:

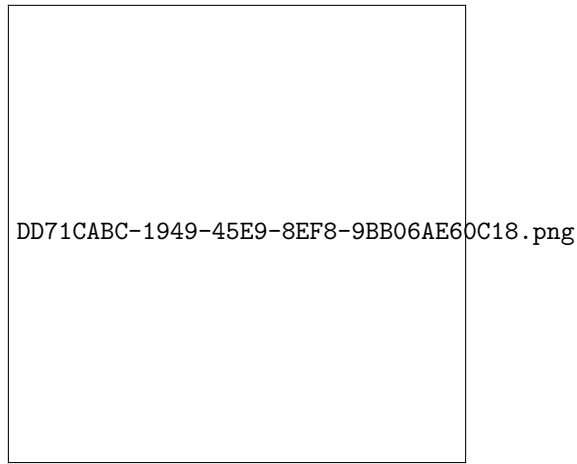


Figure 6: Enter Caption

$O(\log k)$

For n points:

Time Complexity: $O(n \log k + k \log k)$

The final $k \log k$ term comes from sorting the selected points before returning them.

When k is much smaller than n , the heap solution can be more efficient.

Space Complexity: $O(k)$

7 Problem 7: Top K Frequent Values

7.1 1. Specification

Input:

- An integer vector
- An integer k

Output:

The k values with the highest frequency.

If two values have the same frequency, the smaller value must appear first.

Example:

Input:

$[1, 1, 1, 2, 2, 3]$, $k = 2$

Output:

$[1, 2]$

7.2 2. Structure

The function is:

```
topKFrequent(const vector<int>& values, int k)
```

An `unordered_map<int, int>` stores the frequency of each value.

The frequency information is then copied into a vector containing pairs:

```
(value, frequency)
```

The vector is sorted using two rules:

1. Higher frequency comes first.
2. If frequencies are equal, smaller value comes first.

The first `k` elements are returned.

7.3 3. Pseudocode

```
TOP-K-FREQUENT(values, k)
    create empty frequency map
    for each value
        frequency[value]++
    create list items

    for each entry in frequency
        add (value, frequency) to items
    sort items such that
        higher frequency comes first
    if frequencies are equal
        smaller value comes first

return the first k values
```

7.4 4. Correctness

The hash map records the exact number of occurrences of every distinct value.

After sorting:

- Any value with a higher frequency appears before a value with a lower frequency.
- If two frequencies are equal, the smaller value appears first.

Therefore, taking the first `k` values gives exactly the required result according to both the frequency rule and the tie-breaking rule.

For example:

```
4 -> frequency 2
1 -> frequency 2
2 -> frequency 1
```

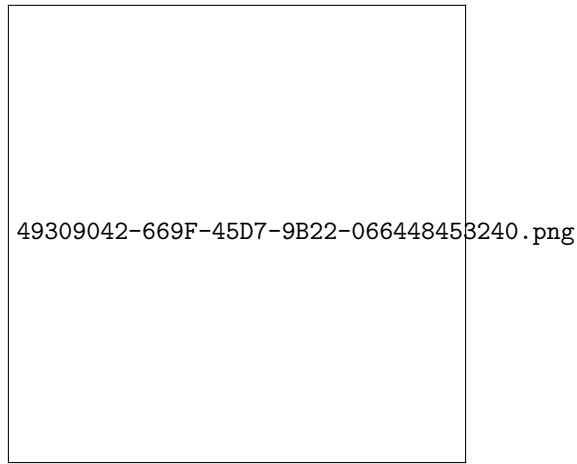


Figure 7: Enter Caption

Because $1 < 4$, the correct order is:
[1,4]

7.5 5. Analysis of Complexity

Let:

- n = total number of input values
- u = number of unique values

Building the frequency table requires:

$O(n)$ average time.

Sorting the u unique values requires:

$O(u \log u)$.

Therefore:

Average Time Complexity: $O(n + u \log u)$

Since $u \leq n$, the worst practical sorting bound can be written as:

$O(n \log n)$.

The frequency map and item vector store at most u entries.

Space Complexity: $O(u)$

8 Problem 8: Merge K Sorted Arrays

8.1 1. Specification

Input:

k individually sorted integer arrays.

Output:

One sorted array containing every value from all input arrays.

The complete output should not be repeatedly sorted.

Example:

Input:

[1,4,5] [1,3,4] [2,6]

Output:

[1,1,2,3,4,4,5,6]

8.2 2. Structure

The function is:

```
mergeKSortedArrays(const vector<vector<int>>& arrays)
```

A min-heap is used.

Each heap element stores:

```
(value, arrayIndex, elementIndex)
```

Initially, the first element of every non-empty array is placed in the heap.

The smallest heap element is repeatedly removed and added to the result.

Then, the next element from the same source array is inserted into the heap.

8.3 3. Pseudocode

```
MERGE-K-SORTED-ARRAYS(arrays)
    create empty minHeap
    for each array i
        if array i is not empty
            insert
                (arrays[i][0], i, 0)
            into minHeap
    result = empty list

    while minHeap is not empty
        remove smallest item
        add its value to result
        nextIndex = elementIndex + 1

        if nextIndex exists
            insert next element
            from the same array
    return result
```

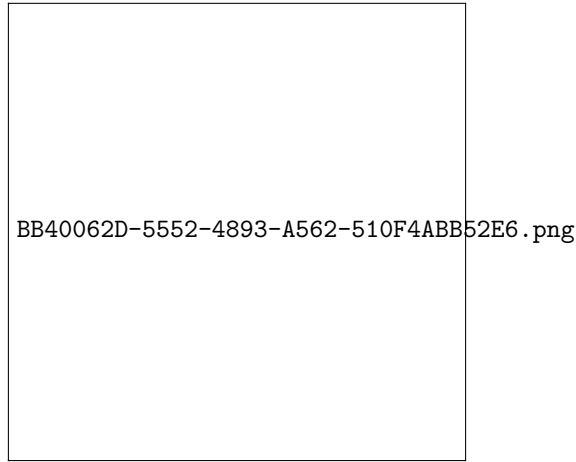


Figure 8: Enter Caption

8.4 4. Correctness

Each input array is already sorted.

The heap contains the smallest unprocessed element from every array that still contains values.

Therefore, the minimum element in the heap is also the smallest unprocessed value among all arrays.

After removing that value, the algorithm inserts the next value from the same array.

This preserves the invariant that the heap always contains the next possible minimum values.

Thus, every element is appended to the result in nondecreasing order, producing a correctly merged array.

8.5 5. Analysis of Complexity

Let:

- k = number of arrays
- N = total number of elements across all arrays

The heap contains at most k elements.

Each of the N values is inserted and removed at most once.

Each heap operation costs:

$O(\log k)$

Therefore:

Time Complexity: $O(N \log k)$

If only one array is present, the algorithm reduces to approximately $O(N)$.

The heap requires:

Auxiliary Space Complexity: $O(k)$

The returned merged array contains N values:

Output Space: $O(N)$

9 Problem 9: Kth Largest Element

9.1 1. Specification

Input:

- An unsorted integer vector
- An integer k

Output:

The k th largest element.

Duplicate values count as separate positions.

Example:

Input:

`[3,2,1,5,6,4]`, $k = 2$

Sorted order from largest to smallest:

`[6,5,4,3,2,1]`

Output:

5

9.2 2. Structure

The solution is implemented in:

```
kthLargest(const vector<int>& values, int k)
```

A min-heap containing at most k values is used.

The heap stores the largest k values encountered so far.

Its smallest value is located at the top.

After all values are processed, that smallest value among the largest k values is exactly the k th largest value.

9.3 3. Pseudocode

```
KTH-LARGEST(values, k)
```

```
    create empty minHeap
```

```
    for each value
```

```
        if heap size < k
```

```
            insert value
```

```
        else if value > heap top
```

```
            remove heap top
```

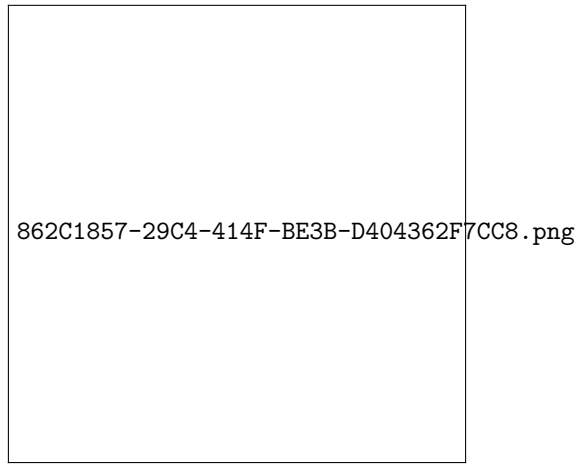


Figure 9: Enter Caption

```
        insert value
    return heap top
```

9.4 4. Correctness

After processing any portion of the array, the heap contains the largest k values encountered so far, or all encountered values if fewer than k have been processed.

If a new value is not larger than the smallest value currently in the heap, it cannot belong to the largest k values.

If it is larger, replacing the smallest heap element maintains the largest k values seen so far.

At the end, the heap contains the largest k values in the entire array.

The smallest of those values, located at the heap top, is exactly the k th largest element.

Duplicate values are stored as separate heap entries, so duplicates correctly count as separate positions.

9.5 5. Analysis of Complexity

The heap contains at most k values.

Each insertion or replacement costs:

$O(\log k)$

For n values:

Time Complexity: $O(n \log k)$

For $k = 1$, heap operations are constant time and the algorithm becomes $O(n)$.

The heap stores at most k values.

Space Complexity: $O(k)$

10 Problem 10: Count Inversions

10.1 1. Specification

Input:

An unsorted integer vector.

Output:

The number of inversion pairs (i, j) satisfying:

$i < j$ and $\text{values}[i] > \text{values}[j]$

The inversion count is stored using a 64-bit integer.

Example:

Input:

[2,4,1,3,5]

Output:

3

The inversion pairs are:

(2,1) (4,1) (4,3)

10.2 2. Structure

The solution modifies Merge Sort.

Three functions are used:

`countInversions()` `mergeSortCount()` `mergeAndCount()`

`countInversions()` creates a temporary vector and starts the recursive Merge Sort.

`mergeSortCount()` recursively divides the array into left and right halves.

`mergeAndCount()` merges two sorted halves while counting cross-inversions.

When:

$\text{values}[i] > \text{values}[j]$

all remaining values from index i through mid are also greater than $\text{values}[j]$.

Therefore, the number:

$\text{mid} - i + 1$

can be added to the inversion count immediately.

10.3 3. Pseudocode

COUNT-INVERSIONS(values)

 create temp array

 return MERGE-SORT-COUNT(

 values,

 temp,

 0,

 n - 1

)

```

MERGE-SORT-COUNT(values, temp, left, right)
    if left >= right
        return 0

    mid = (left + right) / 2
    count = 0
    count += MERGE-SORT-COUNT(
        left half
    )
    count += MERGE-SORT-COUNT(
        right half
    )
    count += MERGE-AND-COUNT(
        left half, right half
    )

    return count

MERGE-AND-COUNT(values, left, mid, right)
    i = left
    j = mid + 1
    inversions = 0
    while both halves have elements
        if values[i] <= values[j]
            copy values[i]
            i++
        else
            copy values[j]
            inversions +=
                mid - i + 1
            j++

    copy remaining values

    return inversions

```

10.4 4. Correctness

The recursive calls correctly count inversions completely inside the left half and completely inside the right half.

The merge step counts inversions where the first element belongs to the left half and the second element belongs to the right half.

Because both halves are sorted, if:

`values[i] > values[j]`

then every remaining element:

`values[i], values[i+1], ... values[mid]`

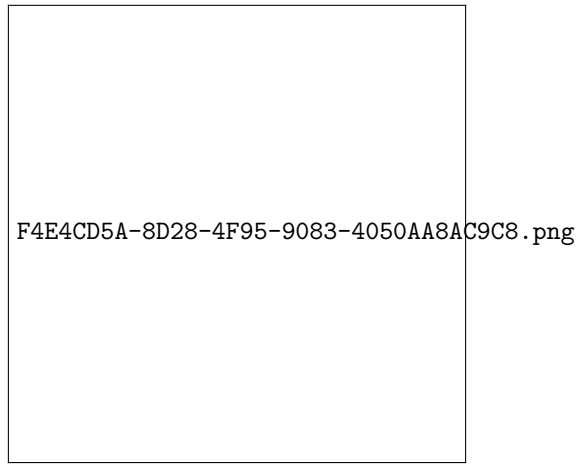


Figure 10: Enter Caption

is also greater than `values[j]`.

Therefore:

`mid - i + 1`

new inversions are created.

Every inversion belongs to exactly one of three categories:

1. Entirely inside the left half
2. Entirely inside the right half
3. Between the left and right halves

The algorithm counts all three categories exactly once.

Therefore, the final result is the correct inversion count.

For the example:

`[2,4,1,3,5]`

the algorithm returns:

3

which corresponds to:

(2,1) (4,1) (4,3)

10.5 5. Analysis of Complexity

Merge Sort divides the problem into two halves at every recursive level.

There are:

$O(\log n)$

levels.

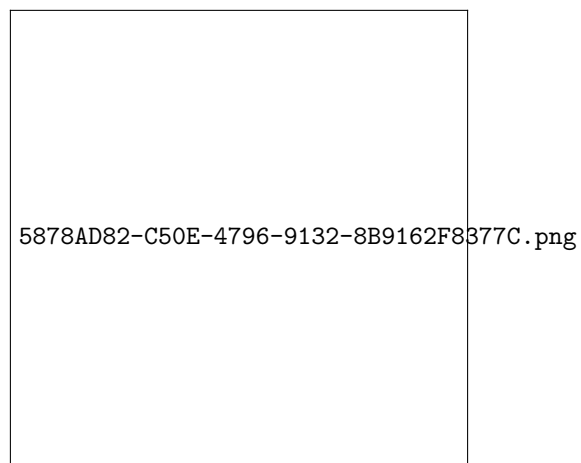


Figure 11: Enter Caption



Figure 12: Enter Caption

At each level, merging all elements requires:

$O(n)$.

Therefore:

Best Case: $O(n \log n)$

Average Case: $O(n \log n)$

Worst Case: $O(n \log n)$

A temporary vector of size n is used during merging.

The recursion stack requires $O(\log n)$ space.

Therefore:

Space Complexity: $O(n)$

A `long long` value is used for the inversion count so that large inversion counts can be stored safely.